

13 00 2560



King Mongkut's University of Technology Thonburi

**Final Exam
CPE334 Operating Systems**

Tuesday May 9th: 13.00-16.00

Instructions

- This is a 12-page exam, please check that you have all the pages before start working.
 - The answers must be written in the space provided in this exam paper. Please read the instruction carefully.
 - A calculator and a paper dictionary are allowed.
 - A single A4-size note make be taken into the examination room (Write on both sides).
-

This examination is designed by Assoc.Prof.Dr. Tiranee Achalakul

Name _____ ID _____



Multiple choices (10 points)

1. Concurrency plays a major part in which of the following specific contexts:
 - a. Multiple applications
 - b. Structured applications
 - c. O/S structure
 - ☒ d. All of the above
2. Examples of solutions to the concurrency problem that do not involve busy waiting are the following:
 - a. Semaphores and monitors
 - b. Message passing and caching
 - c. Producers and consumers
 - ☒ d. None of the above
3. In order to implement mutual exclusion on a critical resource for competing processes, only one program at a time should be allowed:
 - ☒ a. in the critical section of the program
 - b. To perform message passing
 - c. To Exhibit cooperation
 - d. None of the above
4. The following requirement must be met by any facility or capability that is to provide support for mutual exclusion:
 - a. Only one process at a time can be allowed into a critical code section
 - b. A process remains in its critical code section for a finite time only
 - c. No assumptions can be made about relative process speeds
 - ☒ d. All of the above
5. In a uniprocessor system, mutual exclusion can be guaranteed by:
 - a. Overlapping processes
 - b. Interleaving processes
 - ☒ c. Disabling interrupts
 - d. All of the above
6. Which of the following disk seek algorithms has the most variability in response time?
 - a. First Come First Served
 - ☒ b. Shortest Seek Time first
 - c. SCAN
 - d. C-SCAN
7. RAID is a way to
 - a. Increase hard drive latency and performance.
 - b. Increase hard drive performance and decrease cost.
 - ☒ c. Increase hard drive reliability and performance.
 - d. Increase hard drive reliability and decrease cost.

8. Which of the following is not included in an inode in Linux?
 - a. File owner
 - b. File name**
 - c. File modification date
 - d. Pointer to the first data block.
9. Which of the following is not true for about the file system directory hierarchy in Linux?
 - a. Directories are stored in the same way as files
 - b. Directory entries are a fixed size
 - c. It can be represented as a directed graph**
 - d. A single file can have multiple names in the same directory.
10. Which of the following is not a benefit of using larger file blocks in a file system using indexed allocation?
 - a. Better disk space utilization
 - b. Faster access to a randomly-selected block of a large file
 - c. Higher average transfer rate for file data**
 - d. Less disk space consumed for metadata structures

Fill in the blank (10 points)

1. Many approaches to achieving mutual exclusion are software solutions that require the use of a technique called Semaphore.
2. When only one process is allowed in its critical code section at a time, then mutual exclusion is enforced.
3. The technique in which a process can do nothing until it gets permission to enter its critical section but continues to test the appropriate variable to gain entrance is called test-and-set.
4. The situation where Process 1 (P1) holds Resource 1 (R1), while P2 holds R2, and P1 needs R2 to complete and P2 needs R1 to complete is referred to as deadlock.
5. In order to protect shared variables (and other shared global resources) the system must control the Resource Allocation ?.
6. We use the concept of Files and Directories to abstract the file system.
7. The OS needs to create partitions and mount a file system before use
8. When the device has completed the operation, it will raise a hardware interrupt, causing the CPU to jump into the OS at a pre-determined ISR (Interrupt Service Routine)

มีใน
สไลด์

9. DMA (direct memory access)

is a hardware module that it helps the CPU with data movement between main memory and I/O devices

10. We need software in the OS that knows the details of how a particular device works. This type of software is called Device Driver.

Answer the following questions using only the space provided to write your answers. (70 points)

1. (2 points) Given that we can create user-level code to control access to critical sections, why is it important for an operating system to provide synchronization facilities such as semaphores in the kernel?

เพราะมันดีกว่า

Question is about offering sync services via the kernel than via user-level code.

To avoid busy waiting: the waiting thread can go to sleep
→ creates better cpu utilization; avoids priority inversion (if an issue)

NOT:

- The kernel needs to manage critical sections
- Software solutions might be buggy
- Semaphores are useful because ...

จะหลีกเลี่ยง spin-wait โดยให้ thread ของเรา sleep รอคนอื่น

2. (2 points) What is a race condition? Give an example.

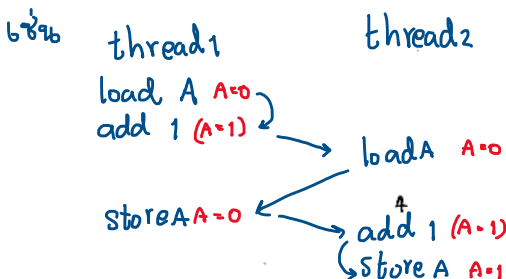
google is my friend

16. What is a race condition? Give an example.

When two (or even more) processes try to concurrently access a shared resource, leading to odd behaviour, deadlocks or mistakenly overwriting memory addresses.

Typical example from lectures: Two processes simultaneously updating a counter (lol, Assignment 1 math.c). Say Process A reads the variable but process B steps in and reads and writes to it. Control is passed back to process A, who just writes to it. You've just written to it twice. Ouch.

เมื่อ thread 2 thread ของเราไปแก้ไขค่าของ shared data ของเราแล้ว



3. (6 points) Compare the two concepts of 'conditional variable' and 'semaphore' by given definition for both terms and explicitly state the differences between the two.

conditional variable

↳ ทำ synchronization

Semaphore

↳ ทำทั้ง mutual exclusion & synchronization



4. (3 points) Give an example (with explanation) of a practical use for a semaphore concept. In your given example, a simple mutex should not be sufficient and a semaphore is then required.

ใช้ mutex ไม่ไหว เพราะ 2 thread ไม่สื่อสารกัน

```
void *producer(void *arg) {
    int i;
    for (i = 0; i < loops; i++) {
        sem_wait(&mutex);
        sem_wait(&empty);
        put(i);
        sem_post(&full);
        sem_post(&mutex);
    }
}

void *consumer(void *arg) {
    int i;
    for (i = 0; i < loops; i++) {
        sem_wait(&mutex);
        sem_wait(&full);
        int tmp = get();
        sem_post(&empty);
        sem_post(&mutex);
        printf("%d\n", tmp);
    }
}
```

5. (2 points) What is a test-and-set instruction? How can it be used to implement mutual exclusion?

21. What is a test-and-set instruction? How can it be used to implement mutual exclusion? Consider using a fragment of pseudo-assembly language aid you explanation.

เป็น atomic

A test and set instruction is a method of using the lock-variable solution, but with hardware support (instructions are in ASM and called before entering any critical region). Prior to entering the critical region, it copies the lock variable to a register, sets it to 1 if it was 0 (free), acquire the lock and execute. If not, another process had it. By being called as ASM instructions, it is guaranteed to be atomic.

เป็น atomic instruction ที่ return ค่าเดิมและอัปเดตค่าใหม่ให้ตัวเอง

```
void lock(lock_t *lock) {
    while (TestAndSet(&lock->flag, 1) == 1);
}
```

เชื่อว่าถ้า flag เป็น 0 ก็เปลี่ยนค่า flag เป็น 1 แล้วหลุด loop ไปยัง lock
ถ้า flag เป็น 1 จะติด spin-wait ไปยัง lock

6. (6 points) The following Java code samples describe two Lock classes with two methods each: `acquire()` and `release()`. You can assume that the application calls `lock.acquire()` before entering a critical section and `lock.release()` after exiting the critical section. For the implementations that require a tid (i.e., thread id), you can assume that the tid of each thread is either 0 or 1.

<pre> class LockA { private int turn = 0; public void acquire(int tid) { while (turn == (1 - tid)); } public void release(int tid) { turn = (1 - tid); } } </pre>	<pre> class LockB { public void acquire() { disableInterrupts(); } public void release() { enableInterrupts(); } } </pre>
--	---

- Does the code guarantee mutual exclusion?
- Does the code guarantee progress?
- List all other limitations that exist for each implementation. Issues you might consider include (but are not limited to) the following: generality, efficiency, and fairness.

Lock A

a) mutual exclusion ได้

b) ไม่ progress

c) ไม่ spin - wait → บัสเวลา

Lock B

a) mutual exclusion ໒

b) ໓ progress

c)

- disableinterrupts (໒) enableinterruptsc) ເປັນ privilege operation
ໃຫ້ user ໃຊ້ໄດ້
- ການປິດ/ປີດ interrupt ໃຫ້ efficiency $\Rightarrow$ ຈຳ
- ໃຊ້ໄດ້ກັບ multi-core

7. (5 points) Suppose there are two processes, A and B, which share semaphores R and S. At the starting time, $R = 0, S = 1$. A and B are entering the sections of code shown below:

$Sem_P = wait$
 $Sem_V = post$

Process A:

```
Sem_P(R);  
Sem_P(S);  
Operation A.1;  
Sem_V(S);  
Operation A.2;  
Sem_V(R);
```

Process B:

```
Operation B.1;  
sem_V(R);  
Operation B.2;  
Sem_P(S);  
Operation B.3;  
Sem_V(S);
```

Each of the following statement, state whether it is true or false and give a brief reason

7.1 Operation A.1 cannot start until operation B.1 is complete.

true เพราะ ถ้า Process A เริ่มทำงานก่อนจะถูกระงับไว้ให้ B ทำงานก่อน

A: Sem-P(R) $\setminus R=0, A=100$

B: B1

ถ้า Process B เริ่มทำงานก่อน B ก็จะไม่เสร็จก่อนจนกว่า A จะเสร็จ

7.2 Operations A.1 and B.2 cannot execute simultaneously.

A: Sem-P(R) $\rightarrow R=0$

B: B1

B: Sem-V(R) $\rightarrow R=1$

A: Sem-P(R) $\rightarrow R=1-1=0$

A: A1

B: B2

false มีโอกาสที่จะพร้อมกันไม่ได้

เพราะ Sem-V() แล้วปลด A มาทำ B ทั่วๆ

ใช้ไม่ได้ให้ B ทั่วๆ



7.3 Operations A.1 and B.3 cannot execute simultaneously

B: Sem-P(R) $\rightarrow R=0$

A: Sem-V(R) $\rightarrow R=1$

A: A1

true เพราะ Sem-P() จะทำใน B ทั่วๆ

จนกว่า A.1 จะทำเสร็จแล้วจึงเรียก Sem-V()

ดังนั้นจะรันมาทำ B.3 ได้

7.4 Operations A.2 and B.3 cannot execute simultaneously

false เพราะ หลังจาก A ใช้ Sem-V() แล้วก็ทำ A.2 ต่อได้

แล้ว Sem-V() ก็จะไปปลด B ให้มาทำ B.3 ต่อด้วย

7.5 The system may enter the deadlock state.

false

8. (2 points) Explain what is meant by *starvation* in the context of a solution to the mutual exclusion problem.

คือ thread ที่เข้าคิว lock แต่ไม่ unlock ทำให้ thread ใ้ไม่ได้ทำงาน

9. (2 points) Briefly describe the concept of producer/consumer (bounded buffer) and give one example of real world application that fits the concept.

producer ៖ check ថា buffer តែងឆ្លើយ → ផ្គត់ផ្គង់ → put
consumer ៖ check ថា buffer រំពឹង → ទទួល → get

10. (2 points) Give a definition of a term "virtual machine".

mid term ១:

11. (3 points) compare and contrast the concepts of "full virtualization" and "para-virtualization".

១២១២២២២

12. (3 points) What are the benefits of virtualization, and what is one situation where you would want to use a VM instead of a physical machine?

mid term ២

13. (2 points) describe the key concept of JAVA virtual machine and its main benefit.

วิธี

14. (3 points) Assuming the operating system detects the system is deadlocked, what can the operating system do to recover from deadlock?

31. Assuming the operating system detects the system is deadlocked, what can the operating system do to recover from deadlock?

ตายแน่

The operating system can recover from deadlock by:

- **Killing the deadlocked process** (crude, and requires that it will be restarted)
- **Rolling Back** (going back a few instructions until it can enter a non-deadlocked state, and retry)
- **Pre-emption** (forcing the handing over of a resource to another process, at the expense of the one it's currently deadlocked on)

หยุดทุก Process ที่เกี่ยวกับ deadlock หรือ หยุด Process ที่จะทำให้หยุด deadlock

15. (7 points) Recall the various deadlock detection and prevention algorithms we've discussed, and consider the following snapshot of a system with five processes (P1, P2, P3, P4, P5) and four resources (R1, R2, R3, R4). There are no current outstanding queued unsatisfied requests.

Currently Available Resources

R1	R2	R3	R4
2	1	2	0

Process	Current Allocation				Max Need				Still Needs			
	R1	R2	R3	R4	R1	R2	R3	R4	R1	R2	R3	R4
P1	0	0	1	2	0	0	3	2	0	0	2	0
P2	2	0	0	0	2	7	5	0	0	7	5	0
P3	0	0	3	4	6	6	5	6	6	6	2	2
P4	2	3	5	4	4	3	5	6	2	0	0	2
P5	0	3	3	2	0	6	5	2	0	3	2	0

- a) Is this system currently deadlocked, or can any process become deadlocked? Why or why not? If not deadlocked, give an execution order.

P_2, P_3, P_4, P_5 มีโอกาสเกิด deadlock

- b) If a request from a process P_1 arrives for (0, 4, 2, 0), can the request be immediately granted? Why or why not? If yes, show an execution order.

ไม่ได้ เพราะ R_2 มี instance เหลือไม่ถึง 4

16. (3 points) Give a scenario where choosing a large filesystem block size might be a benefit; give an example where it might be of disadvantage.

ข้อนี้เลย

36. Give a scenario where choosing a large filesystem block size might be a benefit; give an example where it might be a hinderance.

Large file system block sizes are a help (where performance is concerned) when the files in question are very large, and contiguous reading/writing is prevalent. For example, multimedia files. A large file system block size, unfortunately, wastes a lot of space if your file system largely consists of small sized files. Random access to small segments of data require loading entire blocks of data, even though you only need a small amount of it.

ดี → access ที่เร็ว ได้ข้อมูลเยอะ

ไม่ดี → เปลือง (internal fragmentation)

17. (3 points) What is the reference count field in the inode? You should consider its relationship to directory entries in your answer.

count ว่า มีกี่ directory ที่ reference ถึง file นี้ได้

ข้อนี้

46. What is the reference count field in the inode? You should consider its relationship to directory entries in your answer.

The reference count field is a counter of how many times the inode is "referenced" by name. Adding a directory entry increments this counter. When the count falls to zero, (i.e. there are no longer any directory entries to that file), its inode and all corresponding disk blocks can be safely deallocated.

18. (7 points) In class, we discussed three different techniques for organizing the data blocks for each file in a file system, namely contiguous allocation, linked allocation, and indexed allocation. Briefly describe each approach, identifying the strengths and weaknesses of each

1. Contiguous

1 file มี 1 block ติดกัน

ดี → performance สูง

→ access ได้ทั้ง sequence

เลว = random

ไม่ดี → มี external fragment

→ วนวน file ยาก

2. linked

1 block จะมี pointer ชี้ไป

ที่ block ต่อไปใน file

ดี → ไม่มี external fragment

ไม่ดี → access แบบ random ไม่ได้

3. indexed

1 file มี 1 block ที่เก็บ pointer

ทั้งหมดใน file

ดี → access แบบ random

ไม่ดี → ใช้ pointer เยอะกว่า

19. (2 points) What file allocation strategy is most appropriate for random access files?

contiguous & indexed

แต่แบบ random

indexed →

39. What file allocation strategy is most appropriate for random access files?

An inode (indexed node) based file allocation strategy. It is a form of file allocation table that offers temporal locality – the inode only needs to be located in memory when its corresponding file is open. The inode is a data structure which lists all attributes and points to all the addresses of the disk block corresponding to that file.

20. (5 points) Compare I/O based on polling with interrupt-driven I/O. In what situation would you favor one technique over the other?

เดี๋ยวนี

70. Compare I/O based on *polling* with *interrupt-driven* I/O. In what situation would you favour one technique over the other?

Polling means that we are continually using CPU cycles to check whether any I/O is occurring or not. However, it means the CPU is busy-waiting on any other task so as to not miss any I/O. Interrupt-driven I/O allows the CPU to work on other tasks and handle requests on demand, however it requires a context switch into the interrupt handler to process it. We normally favour interrupt-driven I/O for any sort of human interface device, as it is much slower than the data processing it is doing, so a few hundred interrupts each second won't matter. We might favour polling for high bandwidth raw data transfer applications, as a million context switches tends to slow things down.

polling ใช้กับ device ที่ transfer data เร็ว

ส่วน interrupt ใช้กับ human interface device

